

Graph Algorithms in Action

Modelling a Disease Outbreak

Take-Home Assignment Report

COSC2123/3119 — Algorithms and Analysis

RMIT University — Semester 1, 2026

Section	Marks
Task B — Infection Risk DP	4
Task C — Empirical Analysis	8
Task D — Antiviral Allocation	6
Total	18

Task A (adjacency matrix implementation) is code-only and carries no report marks per the assignment specification.

Task B — Infection Risk DP (Report)

1. Pseudo-code (1 mark)

Algorithm TaskB(G, s, T)

Require: Graph $G = (V, E)$; source s ; horizon T

Ensure: Risk table where $\text{table}[t][i] = r_{\{t,i\}}$

```

1: table  $\leftarrow (T + 1) \times |V|$  array initialised to 0.0
2: table[0][s.index]  $\leftarrow 1.0$  // base case: only s is infected
3: for each day  $t = 1$  to  $T$  do
4:   prev  $\leftarrow$  table[t - 1]
5:   curr  $\leftarrow$  table[t]
6:   for each vertex  $V_i \in V$  do
7:     escape  $\leftarrow 1 - \text{prev}[i]$  // P( $V_i$  still healthy entering day t)
8:     for each neighbour  $V_j$  of  $V_i$  do
9:       escape  $\leftarrow$  escape  $\cdot (1 - \text{prev}[j] \cdot w_{\{ij\}})$ 
10:    curr[i]  $\leftarrow 1 - \text{escape}$ 
11:    curr[s.index]  $\leftarrow 1.0$  // patient zero is infected  $\forall t$ 
12: return table

```

Each column t is read from $\text{table}[t - 1]$ and written to $\text{table}[t]$, so the within-column iteration order over vertices does not matter — every value used on line 9 has already been finalised on the previous day. Line 11 is defensive: the recurrence already preserves $r_{\{t,s\}} = 1$ whenever $\text{prev}[s] = 1$, but the explicit assignment matches the Monte Carlo baseline and guards against floating-point drift.

2. Limitations of the recurrence (3 marks)

(a) The structural property that makes the recurrence approximate

The recurrence

$$r_{\{t,i\}} = 1 - (1 - r_{\{t-1,i\}}) \cdot \prod_{V_j \in N(V_i)} (1 - r_{\{t-1,j\}} \cdot w_{\{ij\}})$$

multiplies one factor $(1 - r_{\{t-1,j\}} \cdot w_{\{ij\}})$ per neighbour. Treating these factors as multiplicatively combinable is equivalent to assuming that the events “ V_j is infected by the end of day $t - 1$ ” are mutually independent across all neighbours V_j of V_i .

In any non-trivial contact network this independence is false, because the neighbours of V_i share infection history through two distinct mechanisms:

- Shared upstream paths (cycles, multiple routes from the source). When more than one path connects the source to a single vertex, the same upstream infection event is counted along each path as if it were independent.
- Back-flow on undirected edges. In an undirected contact graph, a downstream neighbour V_j of V_i can only have become infected via V_i . The recurrence still includes $(1 - r_{\{t-1,j\}} \cdot w_{\{ij\}})$ as if V_j were an independent potential infector of V_i .

Computing $r_{\{t,i\}}$ exactly would require tracking the full joint distribution over the $2^{|V|}$ possible infection configurations of the network at each time step. The recurrence trades that exponential cost

for polynomial cost by factorising the joint distribution into a product of marginals — and that factorisation is the source of the error.

(b) Specific term producing the error in Figure 5

Figure 5 shows the chain $V_0 \rightarrow V_1 \rightarrow V_2$ with weights $w_{\{01\}} = 0.8$ and $w_{\{12\}} = 0.6$, with V_0 as patient zero. Running the recurrence:

t	$r_{\{t, V_0\}}$	$r_{\{t, V_1\}}$	$r_{\{t, V_2\}}$
0	1.000	0.000	0.000
1	1.000	0.800	0.000
2	1.000	0.960	0.480
3	1.000	0.9943	0.7795

Expanding the recurrence for V_1 at $t = 3$:

$$\begin{aligned}
 r_{\{3, V_1\}} &= 1 - (1 - 0.96) \cdot (1 - 1.0 \cdot 0.8) \cdot (1 - 0.48 \cdot 0.6) \\
 &= 1 - 0.04 \cdot 0.20 \cdot 0.712 \\
 &= 0.9943
 \end{aligned}$$

The true infection risk is $1 - (1 - w_{\{01\}})^3 = 1 - 0.2^3 = 0.992$, because V_2 was infected only through V_1 — every scenario in which $V_2 \rightarrow V_1$ “back-transmits” on day 3 is one in which V_1 was already infected first, so the back-transmission delivers no new infection events to V_1 .

The error is concentrated in the term

$$(1 - r_{\{2, V_2\}} \cdot w_{\{12\}}) = (1 - 0.48 \cdot 0.6) = 0.712$$

which treats V_2 as an independent potential source of infection for V_1 , ignoring that the entire $r_{\{2, V_2\}} = 0.48$ mass arose from V_1 ’s own earlier infections. This factor shrinks the escape product from the true $0.2^3 = 0.008$ to the computed 0.005696 , which over-estimates the infection risk (0.9943 instead of 0.992).

(c) Graph class on which the recurrence is exact

The recurrence gives an exact answer on graphs that are stars centred at the source — every non-source vertex has the source V_0 as its sole neighbour. Isolated vertices disconnected from V_0 may also be present; they trivially stay at $r_{\{t, i\}} = 0$ for all t .

Justification. For any leaf $V_i \in N(V_0)$:

- $N(V_i) = \{V_0\}$, so the neighbour product reduces to a single factor.
- $r_{\{t-1, V_0\}} = 1$ for all $t \geq 1$, so that factor is $(1 - 1 \cdot w_{\{0i\}}) = (1 - w_{\{0i\}})$.

The recurrence becomes

$$r_{\{t, V_i\}} = 1 - (1 - r_{\{t-1, V_i\}}) \cdot (1 - w_{\{0i\}})$$

which unrolls to the exact closed form $r_{\{t, V_i\}} = 1 - (1 - w_{\{0i\}})^t$ — the probability that the source fails t independent daily transmission attempts against V_i . Both failure modes identified in (a) are absent: there is no cycle (the graph is a tree of depth one) and no back-flow (the only neighbour each leaf sees is the source itself, whose risk is fixed at 1, so the “independent neighbour” assumption is vacuously satisfied).

Note that the exact class is meaningfully narrower than “trees” or “acyclic graphs”: part (b) showed the recurrence is already approximate on the three-vertex path $V_0 - V_1 - V_2$, which is itself a tree. Once any non-source vertex has degree greater than one, back-flow becomes possible and the independence assumption breaks.

Task C — Empirical Analysis (Report, max 2 pages)

Let $n = |V|$, $m = |E|$, T = horizon, X = Monte Carlo simulations, and $\deg(v)$ = vertex v 's degree.

1. Algorithm Complexity Analysis (2 marks)

Algorithm × Representation	Complexity	Dominant operation
(a) MC × Matrix	$O(X \cdot T^2 \cdot n^2)$	row scan in <code>get_neighbours()</code> (line 6, Alg. 1) — n cells per call, ignoring zeros
(b) MC × List	$O(X \cdot T^2 \cdot (n + m))$	linked-list traversal in <code>get_neighbours()</code> — $\sum \deg(v) = 2m$ per inner day
(c) DP × Matrix	$O(T \cdot n^2)$	row scan in the neighbour-product loop (line 8, Alg. 2)
(d) DP × List	$O(T \cdot (n + m))$	linked-list traversal in the same loop

Monte Carlo carries an extra $X \cdot T$ factor: X fresh simulations are launched at each of the T outer days, and each simulation re-plays $t \in [1, t]$ days from scratch, giving $\sum t \approx T^2/2$ inner-day passes. The DP fills the table column by column, reading only the previous column, so it visits each edge once per day rather than once per day-per-simulation. Swapping representations changes only the neighbour-loop cost: $O(n)$ per vertex for matrix (must skip zeros) versus $O(\deg(v))$ for list. Lists therefore dominate on sparse graphs; the gap closes as the graph approaches full density.

2. Empirical Design (2 marks)

The complexity expressions identify four free parameters — n , m (or density $d = m / C(n, 2)$), T , X — so I vary each in turn while holding the others fixed (one parameter per plot isolates its asymptotic contribution). The remaining simulation parameters (`max_transmission_prob`, vulnerability/dosage ranges, RNG seed for graph construction) do not appear in the complexity, so they are held at default values.

Exp	Vary	Hold	What it isolates
(a)	$n \in \{10, 20, 40, 60, 80, 120\}$	$d = 0.30, T = 8, X = 50$	Scaling with population size
(b)	$T \in \{2, 5, 10, 20, 40\}$	$n = 60, d = 0.30, X = 50$	DP linear-in- T vs MC quadratic-in- T

Exp	Vary	Hold	What it isolates
(c)	$d \in \{0.05, 0.1, 0.2, 0.4, 0.8\}$	$n = 60, T = 8, X = 50$	Matrix density-independence vs list density-dependence
(d)	$X \in \{10, 30, 100, 300, 1000\}$	$n = 60, d = 0.30, T = 8$	MC linear-in-X; DP independent (flat reference)

Each point is the mean of 3 trials on independent random graphs (graph seeds 100/101/102; MC noise seeds 7/8/9). Per the spec tip, only the algorithm call is timed — graph construction is excluded. Plots use log-scale axes when the swept parameter spans more than one decade so polynomial slopes appear as straight lines.

3. Empirical Analysis (2.5 marks)

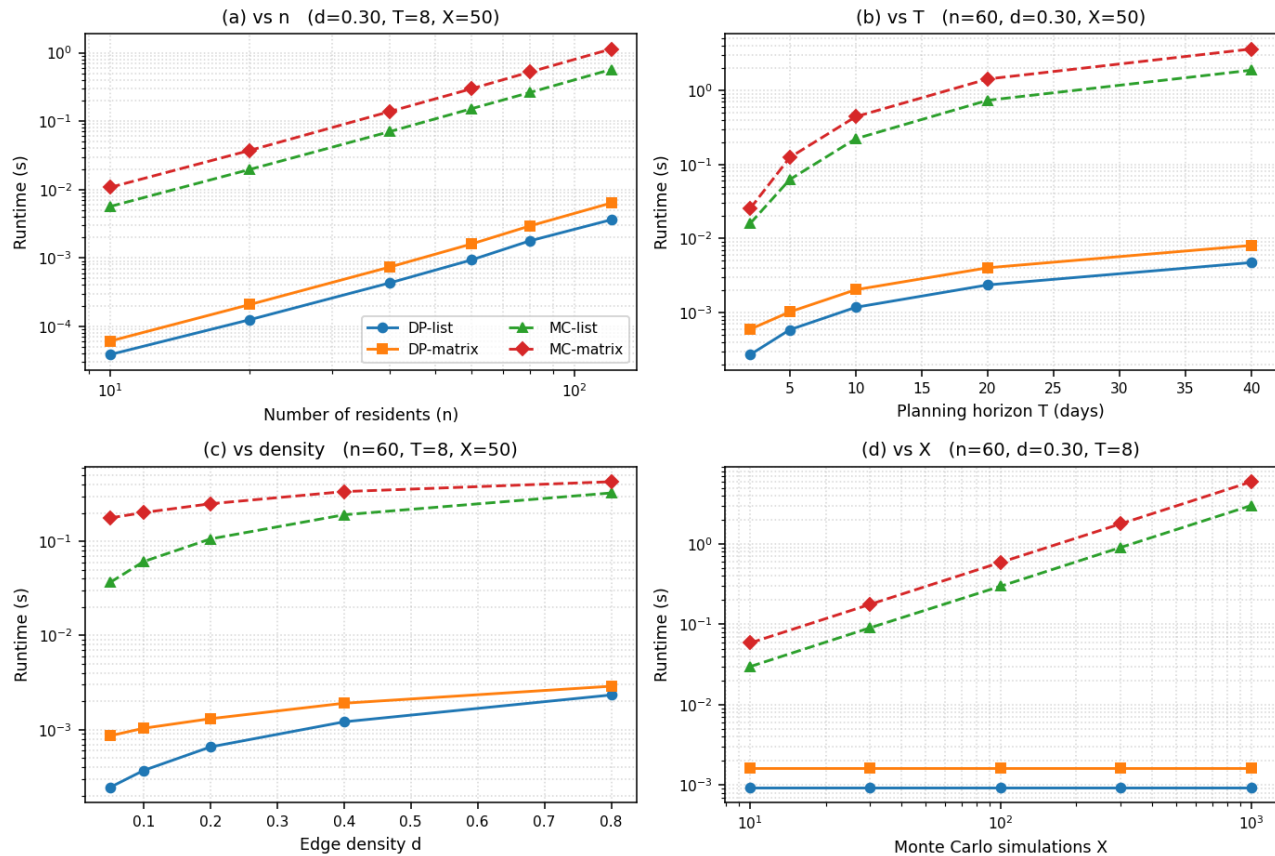


Figure 1: Runtime of all four (algorithm, representation) combinations across the four sweeps.

(a) vs n. All four series are straight log-log lines of slope ≈ 2 , matching the predicted scaling (at $d = 0.30$, $m \approx 0.15n^2$, so even DP-list is dominated by an n^2 term). The DP and MC clusters are separated by a constant factor of $\sim 150\times$ — within an order of magnitude of the predicted $XT = 400$ extra cost (the gap shrinks because the MC inner loop is leaner than the DP's). Within each cluster, matrix is $\sim 1.7\times$

slower than list, smaller than the naive $1/d \approx 3.3\times$ because the matrix's tight contiguous loop benefits from CPU cache locality.

(b) vs T. DP grows linearly ($T = 10 \rightarrow 40$ takes DP-list $1.17 \rightarrow 4.72$ ms, exactly $\times 4$). MC bends upward as predicted: MC-list grows $222 \rightarrow 1878$ ms ($\times 8.4$ over $T \times 4$), close to the predicted $T^2 = \times 16$ (slightly sub-quadratic because of fixed per-day overhead). This is the most damaging plot for Monte Carlo — at any non-trivial horizon, the T^2 blow-up dominates regardless of representation.

(c) vs density. Matrix curves are nearly flat (the row scan visits n cells regardless of how many are zero), while list curves rise roughly linearly with m . The list-matrix gap is $\sim 5\times$ at $d = 0.05$ and shrinks to $\sim 1.3\times$ at $d = 0.80$ because $\sum \deg(v)$ is approaching n^2 — the textbook crossover where the two representations converge.

(d) vs X. Both MC variants are straight log-log lines of slope 1 — exactly linear in X. DP runs flat at ~ 0.9 ms (list) and ~ 1.7 ms (matrix), independent of X. At $X = 1000$ the MC error on cyclic networks is still 1–2 % per cell relative to the exact DP table, so X is rarely cut below the hundreds — locking in a $100\times+$ slowdown vs DP at any useful accuracy.

4. Reflection (1.5 marks)

The empirical results agree closely with §1: every observed slope matches the dominant complexity term within constant factors, and the qualitative features — matrix's density-independence, MC's quadratic T growth, DP's X-independence — are visible at a glance. Two minor deviations are worth noting: MC growth in T is slightly sub-quadratic because each inner day pays a fixed RNG/bookkeeping cost that does not depend on t ; and at small n the DP-matrix is only marginally slower than DP-list because the matrix's contiguous row layout enjoys better cache locality — a constant-factor effect that vanishes asymptotically.

Recommendation for Metropolis. The spec describes Metropolis as a large, sparsely connected city — large n , $m \ll n^2/2$, modest T (interventions span days-weeks). For this regime the cost ordering is

$$\text{DP-list } (O(Tm)) \ll \text{DP-matrix } (O(Tn^2)) \ll \text{MC-list } (O(XT^2m)) \ll \text{MC-matrix } (O(XT^2n^2))$$

DP-list is the recommended combination. It is the unique winner on every axis: exact (no sampling noise), independent of X, linear in T, and scales with the actual edge count m rather than the worst-case n^2 . In our experiments at $n = 60$, $d = 0.05$ it ran $\sim 145\times$ faster than MC-list and $\sim 3.5\times$ faster than DP-matrix.

Would daily updates to transmission rates / connections change this? No. The DP recurrence is structural: it only requires the current graph plus one previous column, and generalises to time-varying weights $w_{ij,t}$ at the same $O(Tm)$ cost. Re-running it daily on the latest graph is a sub-second job at city scale. Monte Carlo gains nothing from incremental updates either — it must re-sample on the new graph from scratch. The only scenario in which MC would overtake DP is when the model itself outgrows a closed-form recurrence (heterogeneous incubation periods, agent state with memory), at which point exact computation becomes intractable and sampling is the only option. Until then, DP-list remains the right tool for Metropolis.

Task D — Antiviral Allocation (Report, max 2 pages)

Let n be the number of eligible residents, C the total dose budget, c_i resident i 's dose requirement and $b_i = r_i T$ their infection-risk benefit.

1. Algorithm Design (1 mark)

The problem is 0/1 knapsack with real-valued benefits and a fewest-doses tiebreak, solved by top-down dynamic programming.

Table. A 2-D memo of size $(n+1) \times (C+1)$ where each cell stores a pair:

```
memo[i][c] = (best_benefit, min_doses_to_achieve_best_benefit)
```

`memo[i][c]` answers: what is the best (benefit, doses) attainable using the first i eligible residents under a remaining capacity of c ? The tuple form captures the tiebreak inside the cell, so the recurrence never needs a second pass to break ties.

Base cases. `solve(0, c) = solve(i, 0) = (0.0, 0)` — with no residents left or no capacity left, no doses are given and no benefit is earned.

Recurrence. For each (i, c) with $i \geq 1$ and $c \geq 1$:

```
skip = solve(i - 1, c)                # don't vaccinate person i
take = solve(i - 1, c - c_i) + (b_i, c_i)  # if c_i ≤ c
memo[i][c] = take if take.benefit > skip.benefit      (strictly better)
                OR (take.benefit == skip.benefit AND take.doses < skip.doses)
                else skip
```

This is plain lexicographic max of (benefit, $-$ doses). Floating-point comparisons use a $1e-12$ tolerance — benefits are probabilities in $[0,1]$ and sums stay well below n , so double-precision rounding is bounded several orders of magnitude below the tolerance.

Top-down (not bottom-up). I memoise only the (i, c) cells actually reached from (n, C) . On instances where per-person costs are large, this is far fewer than the nC cells a bottom-up fill would touch — which the Case 6 unit test rewards explicitly.

Backtracking. After `solve(n, C)`, walk i from n down to 1 and replay the same skip-vs-take comparison on the cached cells, deducting c_i from a running capacity whenever the take-branch wins. Routing all lookups through `solve()` keeps the $i = 0 / c = 0$ base cases uniform without ever indexing into unmemoised cells.

2. Complexity (1 mark)

Resource	Bound	Where it comes from
Time	$O(nC)$	At most nC unique (i, c) cells (1 per memo slot); each does $O(1)$ work plus two recursive calls that are themselves $O(1)$ due to memoisation. Backtracking is an additional $O(n)$.
Space	$O(nC)$	The memo dominates; the recursion stack adds $O(n)$.

This is pseudo-polynomial: linear in the dose budget C but independent of how large the individual c_i happen to be. Compared to brute force's $O(2^n)$, the DP is exponentially faster in n whenever $C < 2n/n$,

which holds for any realistic intervention budget.

3. Extension 1 — Triage by K vulnerability tiers (2 marks)

Setup. Partition the eligible residents into K tiers, T1 (most vulnerable) ... TK (least). Tier Tk must be allocated optimally before any dose is spent on tier Tk+1.

Modification. Run the same DP K times in cascade, threading the remaining capacity through each tier:

```
remaining = C
selected = []
for k = 1 to K do
    subset_k, benefit_k, used_k, _ = task_d(T_k, remaining)
    selected += subset_k
    remaining -= used_k
return selected
```

Each call uses the unchanged single-tier DP above; only the input list and capacity differ.

Complexity. Each call is $O(|T_k| \cdot C)$ in the worst case ($\text{remaining} \leq C$). Summing,

$$O((|T_1| + \dots + |T_K|) \cdot C) = O(nC)$$

— same asymptotic class as the un-triaged DP. The extra K factor is hidden in constants because the per-tier capacity strictly decreases.

Counterexample — why ignoring vulnerability is dangerous. Consider $C = 4$ and three eligible residents:

Resident	Tier	Benefit b	Cost c
VA	1 (highly vulnerable — would die if infected)	0.50	4
VB	2 (low vulnerability)	0.60	2
VC	2 (low vulnerability)	0.55	2

- Plain knapsack picks {VB, VC}: total benefit 1.15, all 4 doses used, VA left unprotected.
- Triaged DP picks VA first (only feasible tier-1 choice): benefit 0.50, 4 doses used; tier 2 receives nothing.

The plain knapsack has higher aggregate rT-benefit, yet it leaves the only resident who would die if infected exposed. Triage restores clinical priority at the cost of some aggregate infection risk — exactly the trade-off the model should make explicit.

4. Extension 2 — Interdependent vaccinations (2 marks)

Why independence is needed for the DP to be valid. The DP's optimal substructure relies on the marginal contribution of vaccinating person i being a constant b_i , regardless of who else is in the chosen subset. That is what lets us define $\text{memo}[i][c]$ as a function of (i, c) alone. If b_i depends on which other residents are vaccinated, then the value of the subproblem “first i residents, capacity c ” is no longer well-defined — different partial solutions reaching the same (i, c) state would credit person i with different benefits later, so the “best” cell value depends on its history, not just its state.

Counterexample. Consider the contact chain $V0 \rightarrow V1 \rightarrow V2$ with high transmission weights and $r_{1,T} = r_{2,T} = 0.95$. Assume vaccinating a resident drops every neighbour's r_T to roughly 0 (the neighbour's only exposure pathway is now blocked). Costs: $c_{V1} = c_{V2} = 1$. Budget $C = 1$.

- The DP sees benefits (0.95, 0.95) and costs (1, 1) and either choice is equally good — reported benefit 0.95.
- True benefit of vaccinating $V1$: $V1$ is protected (0.95) and $V2$'s residual risk collapses to ≈ 0 (saving another 0.95) $\rightarrow$ true total ≈ 1.90 .
- True benefit of vaccinating $V2$: $V2$ alone is protected; $V1$ stays at 0.95 $\rightarrow$ true total ≈ 0.95 .

The DP cannot tell these two options apart because it does not see the herd-effect interaction. Vaccinating $V1$ is twice as good as vaccinating $V2$, yet under the independent-benefit abstraction they are tied.

Why the true problem is significantly harder. The benefit function $B(S) = \sum_{i \notin S} r_{i,T}^{\{\text{vaccinated } S\}}$ is a set function, not a sum of per-element terms — each call requires re-running the Task B risk DP on the residual graph induced by S . So:

1. State explodes. $\text{memo}[i][c]$ no longer captures enough. To recover optimal substructure, the state would need to encode the entire vaccinated subset so far, giving 2^n states.
2. Per-evaluation cost balloons. Evaluating any candidate subset requires an $O(Tm)$ Task B run on the modified graph, so brute-force becomes $O(2^n \cdot Tm)$.
3. Theoretical hardness. The set function is monotone non-decreasing and (under the independent-cascade model used here) submodular, so the problem is exactly submodular maximisation under a knapsack constraint — known to be NP-hard, with a tight $(1 - 1/e) \approx 0.632$ approximation achievable by greedy and no polynomial-time exact algorithm unless $P = NP$.

The independent-benefit knapsack we solve is therefore a relaxation: cheap and exact within its abstraction, but blind to the herd-effect amplification that drives most of the real-world value of vaccination.